

Note de déploiement — TicketTout (Ticket Tout)

Hébergement

Où ce service serait-il hébergé en production ?

OVHCloud, partenaire de l'État, ou un autre hébergeur agréé par l'État. Localisation de préférence en France, ou à défaut en Europe. L'hébergement doit être conforme aux normes de sécurité et de confidentialité des données, notamment le RGPD.

Ressources nécessaires

Quelles ressources cela demanderait-il ?

Exigence de dimensionnement retenue par le client : le service doit être capable d'accueillir **5 000 utilisateurs concurrents** en moyenne. Cette cible est nettement supérieure à ce que suggère le trafic API organique projeté (voir « Estimation CPU/RAM » ci-dessous) — elle est donc traitée comme une contrainte de dimensionnement à part entière (marge de croissance, pics d'usage, montée en charge future), plutôt que déduite du trafic estimé.

Compte tenu du volume de RAM que cette cible implique (voir calcul détaillé plus bas), une architecture **horizontale** (plusieurs instances applicatives derrière un répartiteur de charge) est retenue plutôt qu'un unique serveur surdimensionné :

Ressource	Valeur
Instances applicatives	~10 instances × 8 vCPU / 32 Go RAM, derrière un load balancer
Base de données	Serveur dédié PostgreSQL + PgBouncer (pooling de connexions)
Stockage	100 Go SSD (par instance de base de données, voir « Volumétrie du schéma SQL »)
Système d'exploitation	Ubuntu Server LTS (voir justification ci-dessous)

Le nombre d'instances, leur taille et le dimensionnement de la base de données restent à ajuster selon la charge réelle observée une fois le service en production ; les valeurs ci-dessus sont une proposition de départ cohérente avec la cible de 5 000 utilisateurs concurrents (voir calcul détaillé ci-dessous).

Système d'exploitation

Ubuntu Server LTS (ex. 24.04) est retenu comme système d'exploitation pour l'ensemble des serveurs (applicatifs, base de données) :

- **Léger** : installation serveur minimale sans environnement graphique, empreinte disque et RAM réduite au démarrage.
- **Flexible** : compatible avec l'ensemble de la stack du projet (Docker/Docker Compose, Node.js, PostgreSQL) et disponible nativement chez la plupart des hébergeurs (dont OVHCloud).
- **Maintenu dans la durée** : les versions LTS bénéficient de 5 ans de support et de correctifs de sécurité (jusqu'à 10 ans avec Ubuntu Pro), ce qui limite les migrations d'OS à répétition.
- **Communauté et documentation** : distribution Linux la plus répandue en environnement serveur/cloud, ce qui facilite le support, le recrutement et la résolution d'incidents.

Estimation CPU/RAM

Méthodologie et paliers repris de [colonelserver.com — Server Requirements for High-Traffic Websites](#) :

- **RAM** : RAM totale = RAM de base + (utilisateurs concurrents × RAM par requête), avec 30-100 Mo par requête retenus par la source (**60 Mo en moyenne**).

- **CPU** : paliers indicatifs par niveau de trafic — 2-4 cœurs (blogs/sites vitrine, trafic modéré), 4-6 cœurs (blogs à fort trafic, e-commerce petit/moyen), 8-12 cœurs (grandes plateformes e-commerce, communautés, SaaS), 16+ cœurs (entreprise, load balancing, plusieurs serveurs DB).

Cible retenue : 5 000 utilisateurs concurrents. Cette valeur est directement injectée dans la formule de la source, sans passer par la dérivation à partir du trafic API : pour référence, le trafic organique projeté (~4 022 000 requêtes/mois : 1 000 000 Transaction + 3 000 000 QrCode + 10 000 Users /login + 12 000 Company / Document , voir « Bande passante réseau estimée ») correspond à un débit moyen de $\approx 1,55$ req/s et, avec un facteur de pic $\times 10$ (*hypothèse propre à cette note*), à seulement ≈ 3 à 5 requêtes concurrentes en traitement à un instant donné — très en-deçà des 5 000 utilisateurs concurrents demandés. Les 5 000 utilisateurs concurrents constituent donc une marge de croissance/robustesse volontaire plutôt qu'une simple extrapolation du trafic actuel.

1. **RAM** (formule de la source, RAM de base + utilisateurs concurrents $\times$ RAM/requête) :
 - RAM de base (*hypothèse propre à cette note*) : composants tournant sur chaque serveur d'après docker-compose.yml (Next.js/Node, PostgreSQL, Redis, Garage) ≈ 4 Go.
 - RAM de trafic : 5 000 utilisateurs concurrents $\times$ 60 Mo (moyenne source) ≈ 300 Go (plage source 30-100 Mo/requête $\rightarrow$ 150-500 Go).
 - Total ≈ 304 Go (plage 154-504 Go) si l'on suit la formule de la source à la lettre pour un serveur unique.
 - *Remarque méthodologique* : le ratio de 60 Mo/requête de la source est calibré sur des CMS classiques (rendu HTML + assets par requête), pas sur une API JSON légère comme celle de TicketTout (routes Next.js + requêtes PostgreSQL, réponses de quelques centaines d'octets). Ce chiffre est donc probablement très majorant pour ce projet ; à défaut de profilage mémoire réel par requête, on le conserve par prudence, mais il devra être recalibré avec des métriques applicatives réelles.
2. **Répartition retenue** : plutôt qu'un unique serveur à ~304 Go de RAM, la charge est répartie sur des instances plus petites derrière un load balancer — ex. **10 instances de 500 utilisateurs concurrents** chacune : RAM par instance ≈ 4 Go (base) + 500 $\times$ 60 Mo (30 Go) ≈ 34 Go, arrondi à **32 Go RAM / 8 vCPU** par instance (palier « 8-12 cœurs — grandes plateformes, SaaS » de la source, avec marge).
3. **CPU** (paliers de la source) : le palier « 8-12 cœurs » retenu par instance correspond au trafic d'une plateforme SaaS à fort usage concurrent, cohérent avec la cible de 5 000 utilisateurs simultanés répartie sur plusieurs instances.
4. **Base de données** : 5 000 utilisateurs concurrents peuvent générer jusqu'à 5 000 connexions simultanées si chaque requête ouvre sa propre connexion PostgreSQL, très au-delà du max_connections par défaut de PostgreSQL (100). Un pooler de connexions (**PgBouncer**) est donc nécessaire pour mutualiser un nombre restreint de connexions réelles vers la base (dimensionnement typique : quelques dizaines à ~200 connexions serveur pour des milliers de connexions clientes en mode transaction pooling).

Conclusion : pour tenir la cible de 5 000 utilisateurs concurrents, la configuration initiale (4 vCPU / 8 Go RAM, serveur unique) ne suffit plus — elle reste pertinente uniquement pour le trafic organique actuel (quelques requêtes concurrentes, voir remarque ci-dessus). Le dimensionnement recommandé est une architecture horizontale (~10 instances de 8 vCPU / 32 Go RAM derrière un load balancer, soit ~80 vCPU / 320 Go RAM au total) associée à un pooler de connexions devant la base de données. Ces chiffres restent indicatifs : le ratio RAM/requête de la source est probablement surestimé pour une API JSON légère, et le nombre réel d'instances nécessaires devra être recalibré avec des tests de charge réels (voir remarque sur les tests de bande passante ci-dessous) une fois l'application en production.

Données sortantes

Quelles données sortiraient de l'infrastructure et vers qui ?

Les données sortent de l'infrastructure vers les utilisateurs finaux (salariés, entreprises) et vers les partenaires du service (administrations, partenaires commerciaux).

Volumétrie du schéma SQL

Cette section chiffre le coût de stockage et d'insertion de chaque table définie dans le schéma SQL, afin de dimensionner le stockage et d'anticiper la croissance de la base. N'ayant pas encore de données réelles, les calculs de volumétrie sont basés sur des hypothèses.

Méthodologie et hypothèses

Les calculs se basent sur le modèle de stockage physique de PostgreSQL :

- **En-tête de ligne (heap tuple) :** `HeapTupleHeaderData` = 23 octets, arrondi à **24 octets** avec l'alignement `MAXALIGN` (8 octets).
- **Pointeur de ligne (line pointer)** dans la page : **4 octets**.
- **Champ text / varchar** (varlena) : longueur réelle + **1 octet** d'en-tête pour une chaîne ≤ 127 octets (cas de toutes nos colonnes).
- **Int (int4) :** 4 octets. **Boolean :** 1 octet. **DateTime :** 8 octets. **enum** natif Postgres : 4 octets.
- Un identifiant `String @default(uuid())` est stocké sous forme de texte (36 caractères) : 36 + 1 = **37 octets**.
- Chaque colonne indexée ajoute une entrée de type B-Tree : taille de la clé + **8 octets** (pointeur de tuple `TID`), arrondie à 8 octets.
- Le WAL (Write-Ahead Log) d'une insertion inclut la ligne elle-même, les entrées d'index associées, et un surcoût lié aux « full page images » après checkpoint. On retient un facteur d'amplification **× 2,5** (valeur usuelle observée en écriture aléatoire sur PostgreSQL 15+), à titre d'estimation.

Longueurs moyennes de chaînes retenues pour les colonnes de texte libre :

Colonne	Longueur moyenne estimée
<code>email</code>	25 caractères
<code>name / surname</code>	12 caractères
<code>password</code> (hash bcrypt)	60 caractères
<code>address</code>	35 caractères
<code>postalCode</code>	5 caractères
<code>siret</code>	14 caractères
<code>mimeType</code>	20 caractères
<code>category / reason / name</code> (référentiels)	20-40 caractères
<code>content</code> (QRCode, jeton signé)	180 caractères

Coût d'insertion par table

Table	Données (octets)	Ligne (en-tête+ptr+données)	Index (nb, octets)	Coût insertion (ligne+index)	WAL estimé (×2,5)
<code>CompanyCategory</code>	25 (id 4 + category 21)	56 o	1 (PK) → 16 o	72 o	180 o
<code>CompanyValidationReason</code>	45 (id 4 + reason 41)	80 o	1 (PK) → 16 o	96 o	240 o
<code>Administration</code>	30 (id 4 + name 26)	64 o	1 (PK) → 16 o	80 o	200 o

Document	107 (id 37 + storageKey 37 + mimeType 21 + size 4 + createdAt 8)	136 o	4 → PK 48 + unique(storageKey) 48 + index(storageKey) 48 + index(createdAt) 16 = 160 o	296 o	740 o
Users	219 (id 37 + email 26 + surname 13 + name 13 + role 4 + balance 4 + password 61 + createdAt 8 + updatedAt 8 + expiredAt 8 + documentId 37)	248 o	6 → PK 48 + unique(email) 40 + unique(documentId) 48 + index(role) 16 + index(createdAt) 16 + index(updatedAt) 16 = 184 o	432 o	1 080 o
Company	209 (id 37 + name 21 + email 26 + siret 15 + kbisId 37 + address 36 + postalCode 6 + agentId 4 + reasonId 4 + verified 1 + isFeatured 1 + categoryId 4 + isPartner 1 + createdAt 8 + updatedAt 8)	240 o	9 → PK 48 + unique(email) 40 + unique(siret) 24 + unique(kbisId) 48 + index(verified) 16 + index(createdAt) 16 + index(updatedAt) 16 + index(agentId) 16 + index(reasonId) 16 = 240 o	480 o	1 200 o
Transaction	164 (id 37 + type 4 + userId 37 + companyId 37 + amount 4 + originalTransactionId 37 + createdAt 8)	192 o	6 → PK 48 + unique(originalTransactionId) 48 + index(userId) 48 + index(companyId) 48 + index(type) 16 + index(createdAt) 16 = 224 o	416 o	1 040 o
MinisterFavorite	41 (id 4 + companyId 37)	72 o	1 → unique(companyId) 48 o (△ pas de clé primaire, voir anomalies)	120 o	300 o
QrCode	267 (id 4 + content 181 + expiredAt 8 + userId 37 + companyId 37)	296 o	4 → PK 16 + index(content) 192 + index(userId) 48 + index(companyId) 48 = 304 o	600 o	1 500 o

(o = octets) — WAL (Write-Ahead Log) estimé à 2,5× le coût d'insertion (ligne + index), pour tenir compte des « full page images » et du surcoût d'écriture aléatoire sur PostgreSQL.

Les tables `Transaction` et `QrCode` sont, de loin, les plus coûteuses à l'insertion : `Transaction` du fait de ses trois colonnes UUID indexées, et `QrCode` du fait de l'indexation d'un champ `content` long (jeton signé de ~180 caractères), qui alourdit sensiblement chaque écriture.

Projection de volumétrie annuelle

Hypothèse de charge retenue par le client : environ **10 000 nouveaux employés (Users) par mois**. Les autres tables sont mises à l'échelle selon les ratios déjà établis dans ce document (facteur ×20 par rapport à l'ancienne hypothèse de 500 `Users` /mois). Cette projection ne tient compte que de la croissance brute des tables (insertions), pas des purges/archivages de données :

Table	Volume initial	Croissance mensuelle estimée
Users	8 000	+ 10 000 / mois (hypothèse client : ~10 000 nouveaux employés/mois)
Company	2 000	+ 1 000 / mois (ratio conservé : 10 % du volume de nouveaux Users)
Document	10 000	+ 11 000 / mois (1 document/ Users + 1 KBIS/ Company : 10 000 + 1 000)
Transaction	—	+ 1 000 000 / mois (×20 de l'ancienne hypothèse, proportionnel à la croissance des Users)
QrCode	—	+ 3 000 000 / mois (régénération ~3× par transaction)
Tables référentielles (CompanyCategory, CompanyValidationReason, Administration)	< 100	négligeable

Coût mensuel (ligne + index, hors WAL), avec les coûts d'insertion ci-dessus :

- `Users` : 10 000 × 432 o ≈ 4,32 Mo
- `Company` : 1 000 × 480 o ≈ 0,48 Mo
- `Document` : 11 000 × 296 o ≈ 3,26 Mo
- `Transaction` : 1 000 000 × 416 o ≈ 416 Mo
- `QrCode` : 3 000 000 × 600 o ≈ 1 800 Mo (≈1,8 Go)

Total ≈ 2,22 Go / mois de données (hors index de maintenance et fragmentation), soit environ **26,6 Go / an** en prenant une croissance linéaire du volume mensuel (hypothèse conservatrice qui sous-estime la croissance réelle, `Transaction` et `QrCode` étant cumulatifs et non remplacés — et sans tenir compte d'éventuelles purges).

En ajoutant une marge de 30 % pour le bloat (fragmentation liée à `VACUUM / MVCC`) et les index secondaires non comptés (autovacuum, statistiques), on obtient une estimation d'environ **34,6 Go la première année**.

Le WAL correspondant (~2,5× le volume ci-dessus, soit environ 5,55 Go/mois) n'est pas un coût de stockage permanent : il est recyclé en continu par PostgreSQL, sauf si l'archivage WAL (PITR) est activé, auquel cas il faut prévoir un espace de stockage dédié à l'archive (~66,6 Go/an dans cette hypothèse).

Conclusion : avec cette hypothèse de croissance (×20 par rapport à l'estimation initiale), les 100 Go de stockage SSD prévus dans la configuration serveur restent suffisants sur les 2-3 premières années en croissance linéaire cumulative, mais la marge de sécurité est nettement plus faible qu'avec l'hypothèse précédente (1,65 Go/an). Un plan de scaling du stockage (ou une politique de purge/archivage des tables `Transaction / QrCode` , hors périmètre de cette note) devient pertinent à moyen terme.

Bande passante réseau estimée

En complément du stockage, on estime la bande passante réseau (trafic API, hors assets statiques) nécessaire pour absorber cette croissance. Les tailles ci-dessous ont été **mesurées directement sur l'application en cours d'exécution** (`docker compose --profile dev up, localhost:3000`) via `fetch()` exécuté dans Chrome, le 2026-09-03 :

Endpoint	Corps requête	En-têtes réponse	Corps réponse	Total mesuré (hors en-têtes requête)
<code>GET /health</code>	0 o	229 o	58 o	287 o
<code>POST /api/v1/login</code>	55 o	229 o	32 o	316 o
<code>POST /api/v1/qrcode</code>	100 o	229 o	36 o	365 o
<code>GET /api/v1/employees/:id/balance</code>	0 o	229 o	30 o	259 o

Les en-têtes de requête envoyés par le navigateur (`Host` , `User-Agent` , `Accept-Encoding` , `sec-ch-ua-*` , `Sec-Fetch-*` , etc.) ne sont pas mesurables via l'API `fetch()` elle-même ; on retient une estimation standard de **~650 o** par requête (valeur usuelle pour une requête JSON same-origin sous Chrome), cohérente avec la taille des en-têtes de réponse observée (229 o, générés par Next.js). En sommant requête + réponse, chaque appel API mesuré coûte environ **0,9 à 1,0 Ko** sur le réseau — on retient **1 Ko/requête** comme estimation homogène pour la projection ci-dessous (majorant raisonnable, hors TLS/TCP qui ajoute un léger surcoût supplémentaire).

En associant chaque opération API à la table qu'elle alimente le plus directement (`QrCode` ↔ `/api/v1/qrcode` , `Transaction` /consultation de solde ↔ `/api/v1/employees/:id/balance` , `Users` ↔ `/api/v1/login`), et avec la croissance mensuelle de l'étape précédente :

- `Transaction` : 1 000 000 requêtes/mois × 1 Ko ≈ **1,0 Go/mois**
- `QrCode` : 3 000 000 requêtes/mois × 1 Ko ≈ **2,9 Go/mois**
- `Users` (création/connexion) : 10 000 requêtes/mois × 1 Ko ≈ 10 Mo/mois (négligeable)
- `Company` + `Document` : ~12 000 requêtes/mois × 1 Ko ≈ 12 Mo/mois (négligeable)

Total ≈ 4 Go / mois de trafic API, soit une moyenne sustained d'environ **1,5 Ko/s** sur le mois (les pics réels seront très supérieurs à la moyenne, mais restent négligeables face à la capacité d'une liaison serveur standard).

Conclusion bande passante : même avec l'hypothèse haute de 10 000 nouveaux employés/mois, ~4 Go/mois de trafic API est négligeable comparé à la capacité d'une liaison dédiée classique (une simple connexion 100 Mbps offre une capacité théorique d'environ 32 To/mois à pleine charge). Même en imaginant les 5 000 utilisateurs concurrents ciblés déclenchant chacun une requête au même instant, la charge instantanée resterait de l'ordre de 5 000 × 1 Ko ≈ 5 Mo — négligeable. La bande passante réseau n'est donc pas un facteur limitant pour ce projet ; le stockage disque et le dimensionnement CPU/RAM/connexions base de données (voir « Estimation CPU/RAM ») restent les axes de dimensionnement à surveiller en priorité.

Limite de ces mesures : les tests de bande passante ci-dessus ont été réalisés à un stade du projet où seuls 4 endpoints étaient disponibles (`/health` , `/api/v1/login` , `/api/v1/qrcode` , `/api/v1/employees/:id/balance`), et sans jeu de données réel (réponses d'erreur pour la plupart des appels testés, faute d'API opérationnels à ce stade). Ces mesures sont donc représentatives de la structure générale des réponses JSON de l'application, mais pas forcément de la taille de payload de toutes les routes prévues (listes paginées, tableaux de bord, historiques de transactions, etc., qui pourront renvoyer des réponses plus volumineuses). Il sera nécessaire de refaire ces tests une fois le projet suffisamment avancé pour disposer de davantage d'endpoints implémentés et de données réelles, afin d'obtenir une estimation de bande passante plus représentative.